

Data Structures

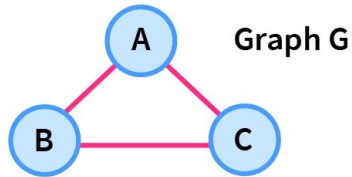
Lecture 23: Minimum Spanning Tree

Soobin Um

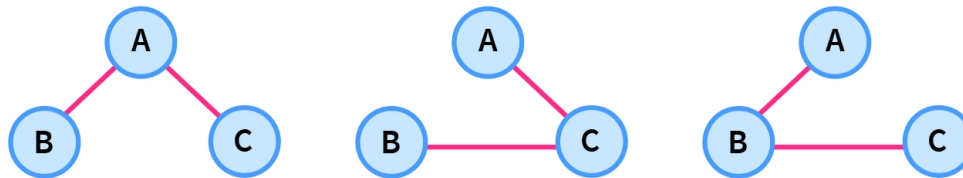
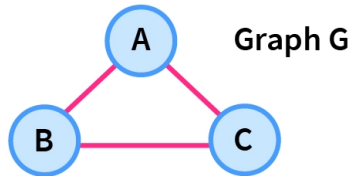
Jun. 8, 2026

Minimum Spanning Tree Problem

Spanning Tree (신장 트리)

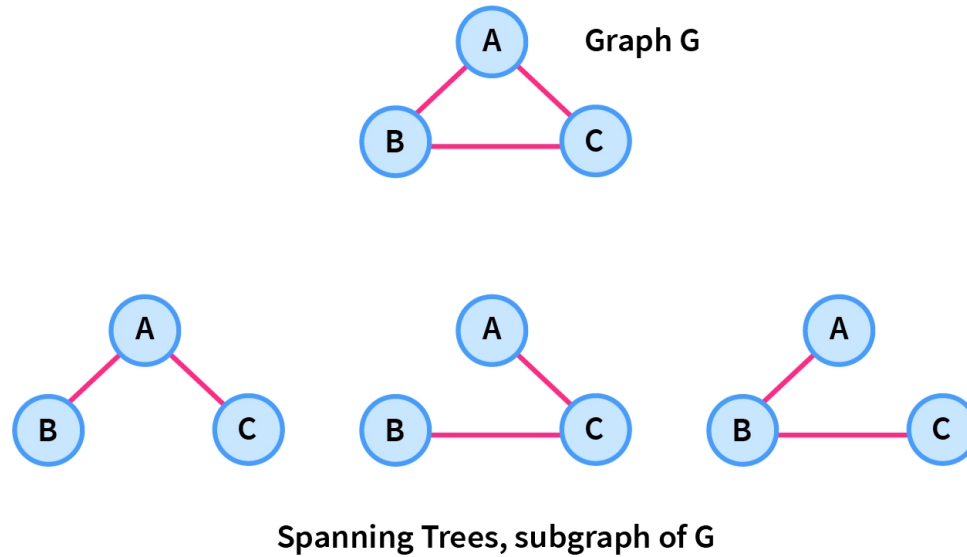


Spanning Tree (신장 트리)



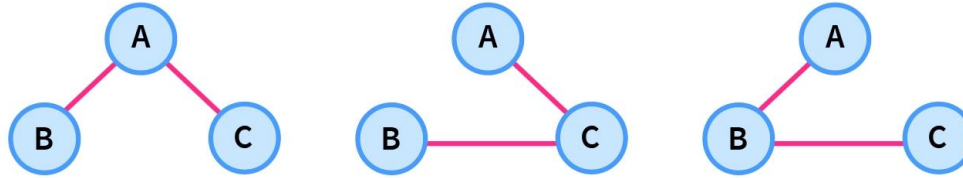
Spanning Trees, subgraph of G

Spanning Tree (신장 트리)



- **신장 트리:** 그래프의 모든 정점을 포함하는 트리 형태의 부분 그래프
 - Spanning: 모든 정점을 아우르는 (포함하는)
 - Tree: 사이클이 없는 연결 구조

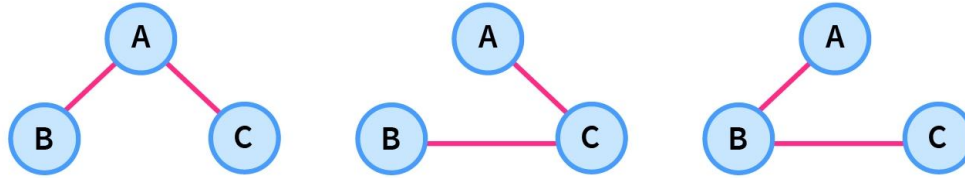
Minimum Spanning Tree (최소 신장 트리)



Spanning Trees, subgraph of G

- **신장 트리**: 그래프의 모든 정점을 포함하는 트리 형태의 부분 그래프
 - Spanning: 모든 정점을 아우르는 (포함하는)
 - Tree: 사이클이 없는 연결 구조
- **최소 신장 트리**
 - Spanning tree 중 간선 **가중치의 총합이 최소**가 되는 트리

Minimum Spanning Tree (최소 신장 트리)



Spanning Trees, subgraph of G

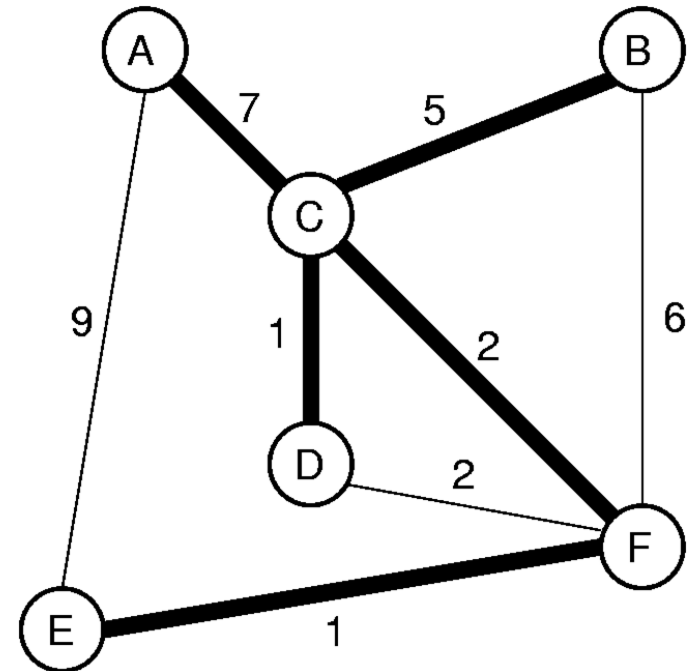
- **신장 트리**: 그래프의 모든 정점을 포함하는 트리 형태의 부분 그래프
 - Spanning: 모든 정점을 아우르는 (포함하는)
 - Tree: 사이클이 없는 연결 구조
- **최소 신장 트리**
 - Spanning tree 중 간선 **가중치의 총합이 최소**가 되는 트리

최소 신장 트리 문제

= 최소 비용으로 모든 정점을 연결하는 방법을 찾는 문제

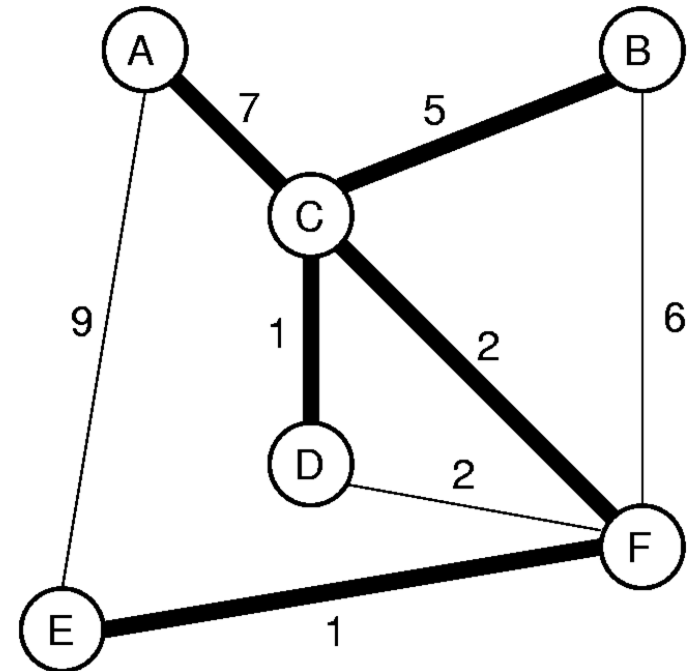
Minimum Spanning Tree Problem

- 입력: 가중치가 있는 **무방향** 연결 그래프
- 출력: **최소 신장 트리** (Minimum Spanning Tree, MST)
 - MST: 간선 **가중치의 총합이 최소**가 되는 트리
- 대표 알고리즘: **Prim**, Kruskal



Minimum Spanning Tree Problem

- 입력: 가중치가 있는 **무방향** 연결 그래프
- 출력: **최소 신장 트리** (Minimum Spanning Tree, MST)
 - MST: 간선 **가중치의 총합이 최소**가 되는 트리
- 대표 알고리즘: **Prim**, Kruskal
- **최소 신장 트리의 특징**
 - (정의) 간선 **가중치의 총합이 최소**
 - 사이클이 없음
 - 간선의 수: $|V|-1$



Prim's Algorithm

Prim's Algorithm

- 무향 가중치 그래프의 MST를 찾아주는
그리디(greedy) 알고리즘



Robert C. Prim (1921–2021)
수학자 / 컴퓨터 과학자

Prim's Algorithm

- 무향 가중치 그래프의 MST를 찾아주는 그리디(greedy) 알고리즘
- 체코 수학자 Vojtěch Jarník가 1930년 최초 고안



Robert C. Prim (1921–2021)
수학자 / 컴퓨터 과학자

Prim's Algorithm

- 무향 가중치 그래프의 MST를 찾아주는 그리디(greedy) 알고리즘
- 체코 수학자 Vojtěch Jarník가 1930년 최초 고안
- 미국 수학자 Robert C. Prim⁰이 1957년 재발견



Robert C. Prim (1921–2021)
수학자 / 컴퓨터 과학자

Prim's Algorithm

- 무향 가중치 그래프의 MST를 찾아주는 그리디(greedy) 알고리즘
- 체코 수학자 Vojtěch Jarník가 1930년 최초 고안
- 미국 수학자 Robert C. Prim이 1957년 재발견
- 네덜란드 수학자 Edsger W. Dijkstra도 1959년 제안



Robert C. Prim (1921–2021)
수학자 / 컴퓨터 과학자

Prim's Algorithm

- 무향 가중치 그래프의 MST를 찾아주는 그리디(greedy) 알고리즘
- 체코 수학자 Vojtěch Jarník가 1930년 최초 고안
- 미국 수학자 Robert C. Prim이 1957년 재발견
- 네덜란드 수학자 Edsger W. Dijkstra도 1959년 제안
- 이들의 초성을 딴 DJP 알고리즘으로도 불림



Robert C. Prim (1921–2021)
수학자 / 컴퓨터 과학자

Setup

- S: 이미 MST에 포함된 정점들의 집합
(즉, 현재까지 트리로서의 구성이 확정된 정점들)
 - 초기화: $S = \emptyset$
- D: 비용 배열
 - $D[i]$: 현재까지 알려진 트리 S와 정점 i를 연결하는 최소 비용 간선의 가중치
 - 초기화: $D[s] = 0$, 나머지 정점들에 대한 거리 값은 ∞

Procedure

S: MST 정점 집합
D: 비용 배열

- **Step 1. Select Min-Distance Vertex**
 - 집합 $V - S$ 중, $D[u]$ 값이 가장 작은 정점 u 를 선택
 - 정점 u 를 S 에 추가 ($\rightarrow$ MST 구성 정점 확정)

Procedure

S: MST 정점 집합
D: 비용 배열

- **Step 1. Select Min-Distance Vertex**
 - 집합 $V - S$ 중, $D[u]$ 값이 가장 작은 정점 u 를 선택
 - 정점 u 를 S 에 추가 ($\rightarrow$ MST 구성 정점 확정)
- **Step 2. Update Costs for Each Neighbor**
 - 선택된 정점 u 의 각 이웃 v 에 대해:
 - $\rightarrow u$ 를 통해 연결되는 간선 (u, v) 의 가중치 w 가 현재의 $D[v]$ 보다 작다면, $D[v]$ 를 w 로 갱신

UpdateCosts (u, v, w):
 $w = \text{weight}(u, v)$
if $w < D[v]$:
 $D[v] = w$

Procedure

S: MST 정점 집합
D: 비용 배열

- **Step 1. Select Min-Distance Vertex**

- 집합 $V - S$ 중, $D[u]$ 값이 가장 작은 정점 u 를 선택
- 정점 u 를 S 에 추가 ($\rightarrow$ MST 구성 정점 확정)

- **Step 2. Update Costs for Each Neighbor**

- 선택된 정점 u 의 각 이웃 v 에 대해:
 - $\rightarrow u$ 를 통해 연결되는 간선 (u, v) 의 가중치 w 가
현재의 $D[v]$ 보다 작다면, $D[v]$ 를 w 로 갱신

UpdateCosts (u, v, w):

$w = \text{weight}(u, v)$

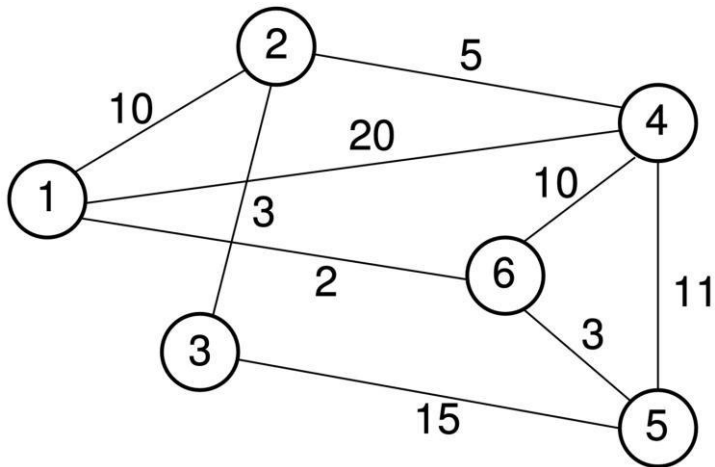
if $w < D[v]$:

$D[v] = w$

- **Step 3. Termination**

- 모든 정점이 S 에 포함될 때까지 Steps 1 & 2를 반복
- 선택된 $|V|$ 개의 정점과 $|V|-1$ 개의 간선이 MST를 구성

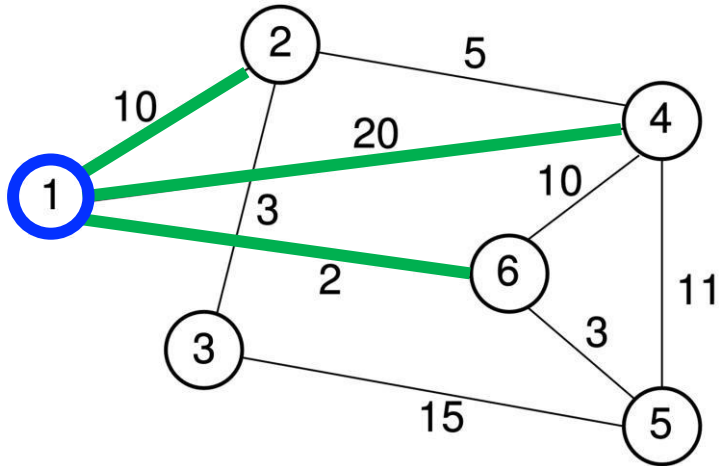
Prim's Algorithm



Distance Array D

	1	2	3	4	5	6
Init.	0	∞	∞	∞	∞	∞
1						
2						
3						
4						
5						
6						

Prim's Algorithm

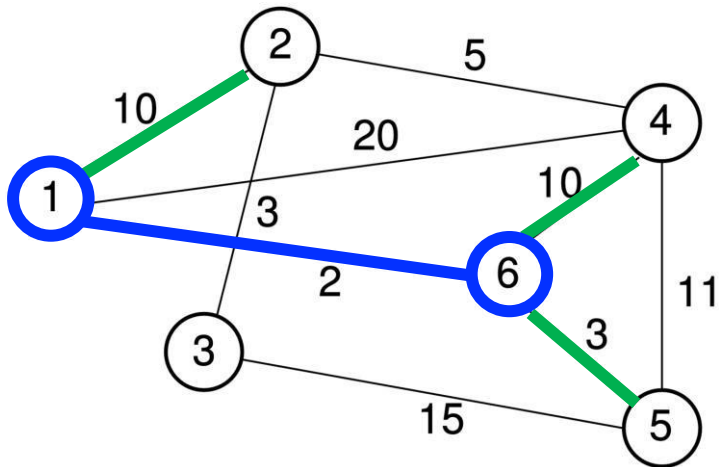


Distance Array D

	1	2	3	4	5	6
Init.	0	∞	∞	∞	∞	∞
1	0	10	∞	20	∞	2
2						
3						
4						
5						
6						

 MST 포함이 확정된 정점들의 집합 S

Prim's Algorithm

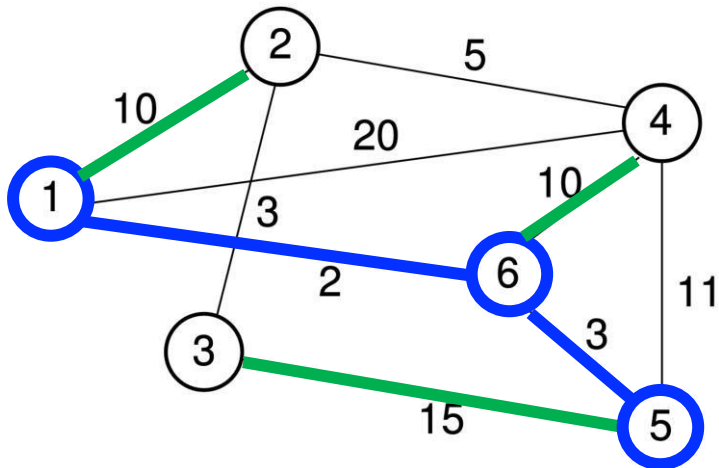


Distance Array D

	1	2	3	4	5	6
Init.	0	∞	∞	∞	∞	∞
1	0	10	∞	20	∞	2
2	0	10	∞	10	3	2
3						
4						
5						
6						

MST 포함이 확정된 정점들의 집합 S

Prim's Algorithm

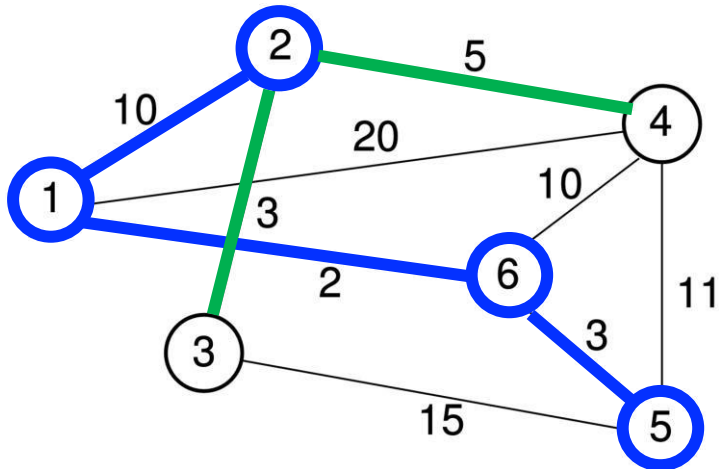


Distance Array D

	1	2	3	4	5	6
Init.	0	∞	∞	∞	∞	∞
1	0	10	∞	20	∞	2
2	0	10	∞	10	3	2
3	0	10	15	10	3	2
4						
5						
6						

MST 포함이 확정된 정점들의 집합 S

Prim's Algorithm

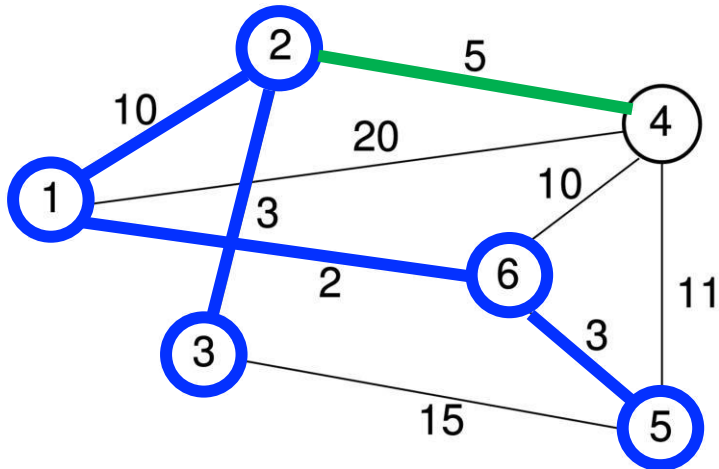


Distance Array D

	1	2	3	4	5	6
Init.	0	∞	∞	∞	∞	∞
1	0	10	∞	20	∞	2
2	0	10	∞	10	3	2
3	0	10	15	10	3	2
4	0	10	3	5	3	2
5						
6						

MST 포함이 확정된 정점들의 집합 S

Prim's Algorithm

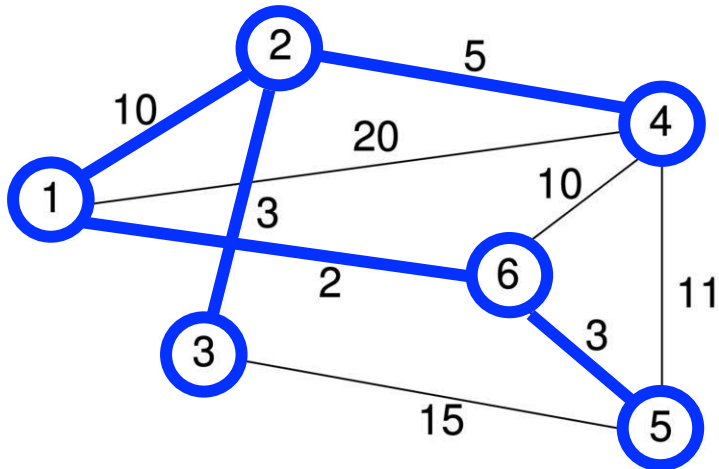


Distance Array D

	1	2	3	4	5	6
Init.	0	∞	∞	∞	∞	∞
1	0	10	∞	20	∞	2
2	0	10	∞	10	3	2
3	0	10	15	10	3	2
4	0	10	3	5	3	2
5	0	10	3	5	3	2
6						

MST 포함이 확정된 정점들의 집합 S

Prim's Algorithm



MST

Distance Array D

	1	2	3	4	5	6
Init.	0	∞	∞	∞	∞	∞
1	0	10	∞	20	∞	2
2	0	10	∞	10	3	2
3	0	10	15	10	3	2
4	0	10	3	5	3	2
5	0	10	3	5	3	2
6	0	10	3	5	3	2

MST 포함이 확정된 정점들의 집합 S

Pseudocode

```
function Prim(G, r):  
    S = {r}  
    MST = {}  
    initialize an array D of size |V| with infinity  
    initialize an array prev of size |V|  
  
    D[r] = 0  
    while S != V:  
        u = minNode(V-S, D)  
        add u to S  
        add edge (prev[u], u) to MST  
        for v in G.neighbors(u):  
            if v is not in S and w(u,v) < D[v]:  
                D[v] = w(u,v)  
                prev[v] = u  
  
    return S, MST
```

시간 복잡도 (== Dijkstra's Algorithm)

- **방법 1: Distance Table (배열) 스캔을 통해 최소 비용 정점 선택 시**
 - 매 단계마다 거리 배열 D 를 앞에서부터 끝까지 선형 탐색
 - 방문 되지 않은 정점 중 D 값이 최소인 정점 찾음
 - **비용: $O(|V|^2 + |E|) = O(|V|^2)$**
 - 정점 V 번 선택 후, 매번 $O(|V|)$ 회의 검색 수행 $\rightarrow O(|V|^2)$
 - $|E|$ 회의 relaxation 발생 $\rightarrow O(|E|)$
- **방법 2: Min-Heap 사용 (표준)**
 - 방문 되지 않은 정점들을 min-heap (priority queue)에 저장
 - D 값이 가장 작은 정점을 바로 추출 가능
 - **비용: $O((|V| + |E|) \cdot \log |V|)$**
 - 초기 buildHeap 1회 $\rightarrow O(|V|)$ (무시됨)
 - 매 정점에 대해서 extract-min 필요 $\rightarrow O(|V| \cdot \log |V|)$
 - 간선마다 relaxation 발생 시 heap 업데이트 필요 $\rightarrow O(|E| \cdot \log |V|)$

Look ahead

Final Exam